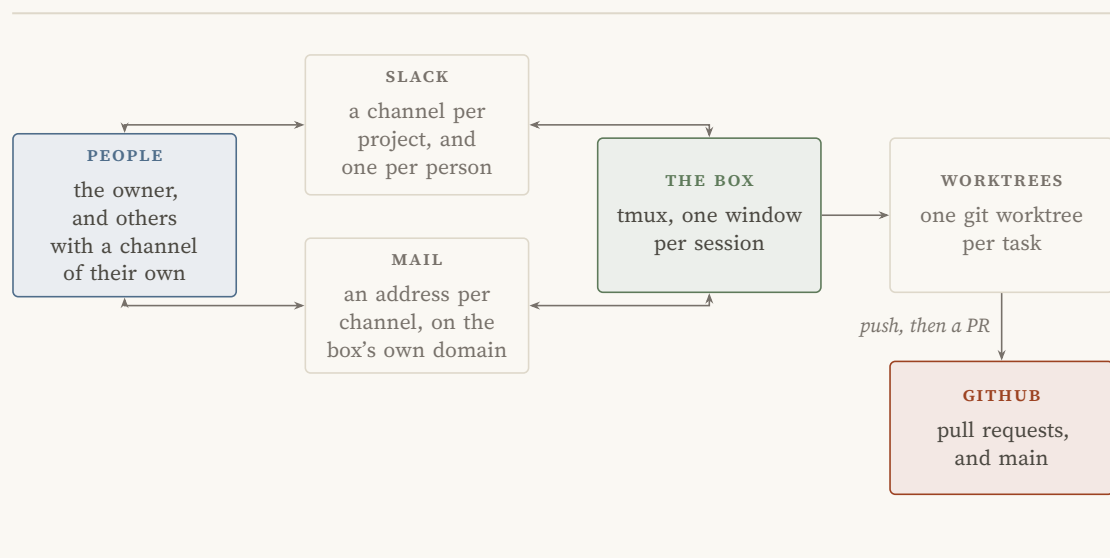


A deeper look

How the box works, one level down from the pitch

The pitch says what this is. This one is for someone who builds software and wants to know how it works. Every figure was read off the box on 21 September 2026, and the grey line at the foot of each page says where it came from.

FIGURE 1 *where work comes in, and where it goes*



Slate is the people, grey the two ways in, sage the machine and its sessions, and rust where work becomes public. A double arrow carries messages both ways.

The machine is one used mini desktop that stays on, with no screen and no keyboard. Every session is a Claude Code process running in a tmux window. Every project is a git repo, and every task gets a git worktree of its own, so two agents never edit the same checkout.

Work comes in two ways, and Slack is the main one. You can also mail an address on the box's own domain, which reaches the same channels, so someone who never opens Slack can send a request in and get the work back. The owner can also ssh in and type into the same tmux windows.

The models run at the AI provider, so the machine itself can be small. What lives here is the layer around them, deciding who may do what, how work is checked, what it costs and what happens when an agent is wrong. That layer is 62 small tools, about 65,000 lines. The generic half is published as open source under the name autobox, and this box's own configuration stays private.

Five kinds of session

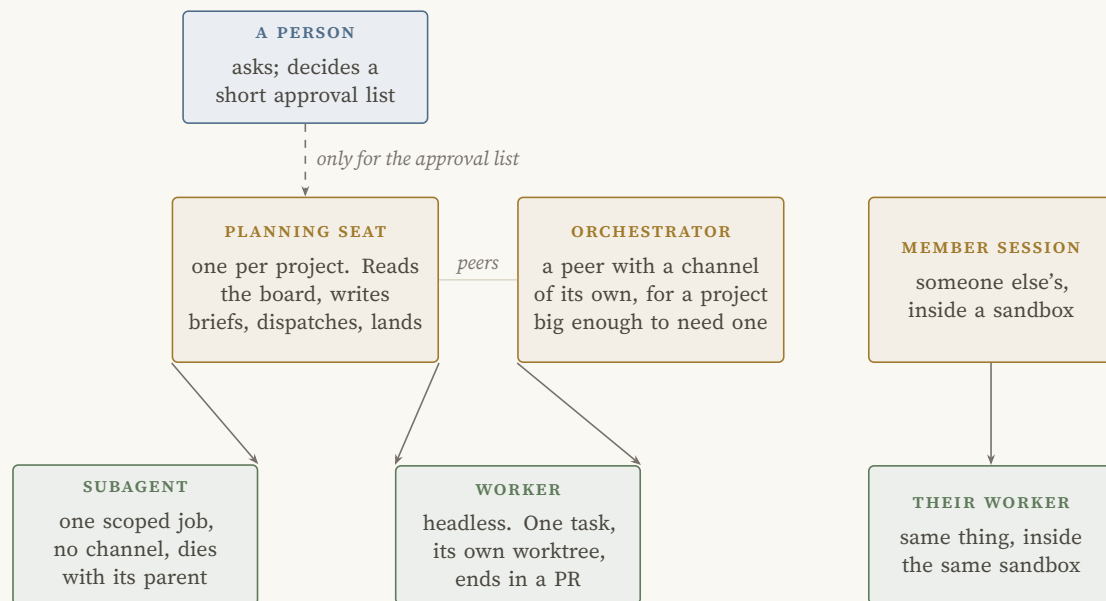
All five run the same agent. What changes is what each one is allowed to do and how long it lives.

A *planning seat* is the long-lived session for one project. It reads the board, writes the briefs, hands work out and lands it. It never commits on the default branch itself, so its own small fixes go on a branch and through a pull request like anything else.

A *worker* runs with nobody watching it. It gets one task, its own worktree and branch, and a fresh context each time round the loop. It ends by opening a pull request, and it never merges one. A hook commits and pushes for it, so a worker that gets interrupted loses nothing.

A *subagent* is the cheap option. It does one read or edit inside another session's turn, it has no channel and no row on the board, and it dies with the session that started it. An *orchestrator* is a planning seat's peer, for a project with enough going on to want its own channel. A *member session* belongs to someone other than the owner, and it can only reach one folder (page 10).

A brief says what the goal is, why it matters, where the boundaries are and how to tell when it is done. It does not say how, because the worker designs the change and its tests itself.

FIGURE 2 *the five kinds, and who starts whom*

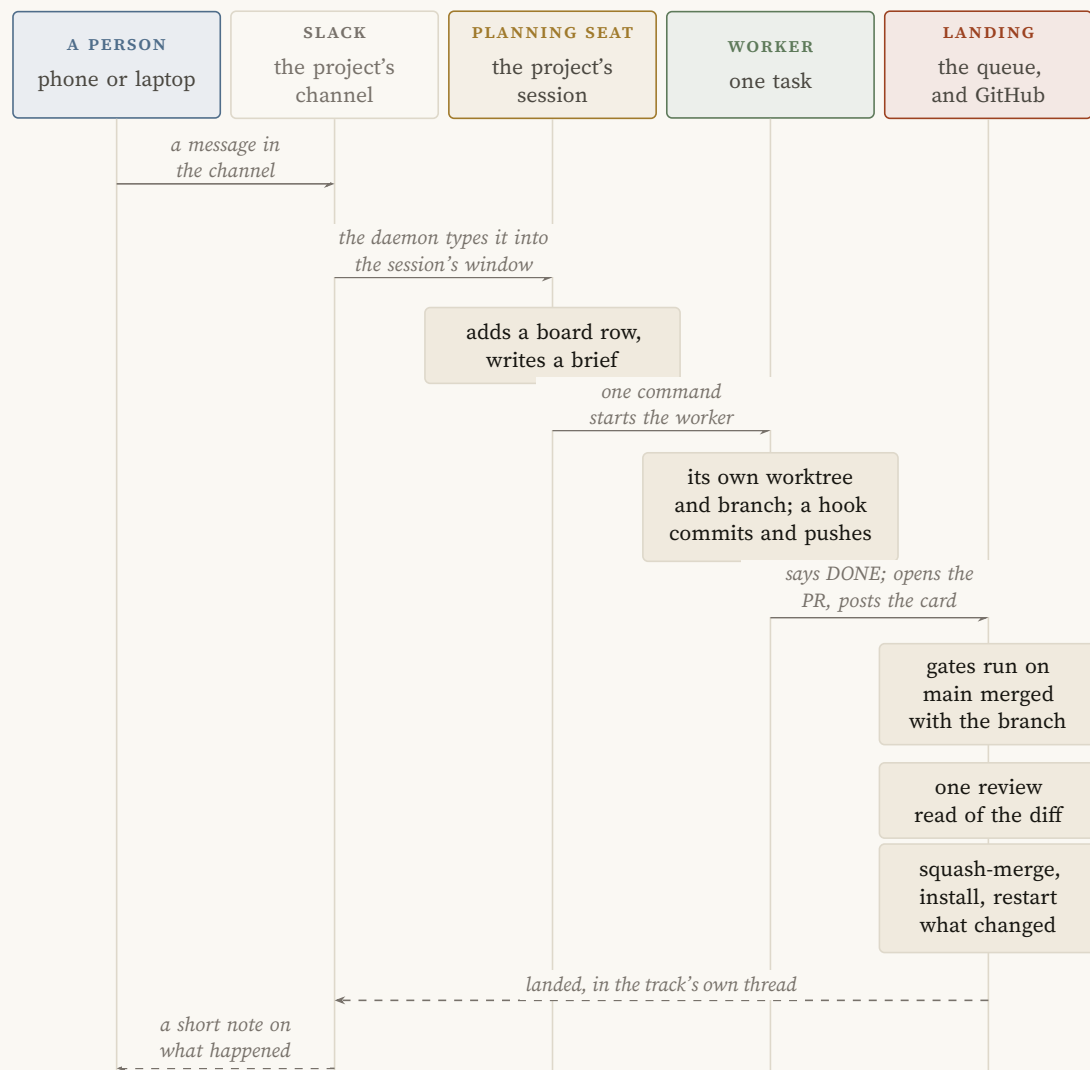
Slate is a person, ochre the long-lived sessions that hold a channel, and sage the short-lived ones. A solid arrow starts a session, the dashed one asks a person.

One request, end to end

A message in a channel turns into a branch, then a pull request, then a merge, and then a change that is running. No person touches any of the steps in between.

The Slack daemon types the message straight into the session's tmux window. Nothing queues it on the way, and nothing else decides what the session gets to see.

The planning seat adds a row to the board and writes the brief, then starts a worker with one command. The worker runs a set number of rounds, each with its own budget and a fresh context that re-reads the journal it keeps, so a long task never drags a long conversation behind it.

FIGURE 3 *one request, end to end*

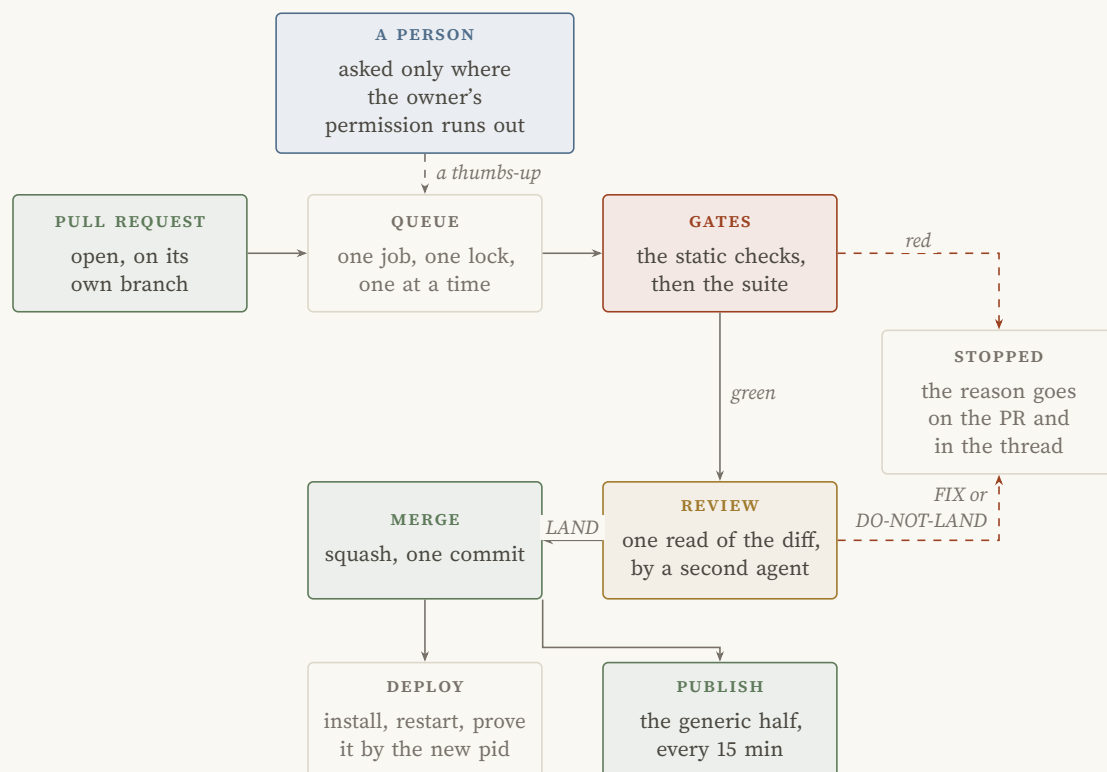
One column per actor, time running down: slate the person, grey Slack, ochre the planning seat, sage the worker, rust the landing. Italic says what crosses; dashed is the way back.

When the worker says it is done, its pull request joins the landing queue. Some projects have the owner's standing permission and merge their own pull requests once everything passes. The rest wait for a thumbs-up on a card.

How a change gets in

Every pull request in every project goes through one queue and the same path. It can be stopped in two places, by a check that fails or by the review that follows.

FIGURE 4 *the one path every pull request takes*



Slate is a person, sage the change as it moves, grey the machinery, rust the gates and ochre the review. The rust dashed lines are the two ways a change is stopped.

The gates run on the default branch with the change merged into it, in a throwaway worktree, because what gets merged is what has to be tested. A change to documents alone runs the cheap static checks. A code change also runs the suite, handed the paths it touched. A wide change, or anything in the tests, the installer or the config, runs everything.

Gate list and tiering: `core/bin/cc-land` (GATES, SUITE). Publish cadence: `cc-publish.timer`, every 15 min, one commit per publish, refused outright if the tree names this box.

The same rule decides whether a change is worth paying a model to read, and only a few kinds are: the control layer, a wide change, the tests, the installer and the config. A change gets one review, keyed to the change itself rather than the branch, so pushing again does not buy a second read. Past one review and one round of repairs, the landing stops and asks a person.

444	152	97	28
pull requests reached this queue since 1 September	of them were stopped at least once before they merged	of those the review sent back, asking for a fix or refusing them outright	failed a gate

The rest hit a conflict, ran out of review budget, or came back from the review with no verdict at all.

What runs with nobody there

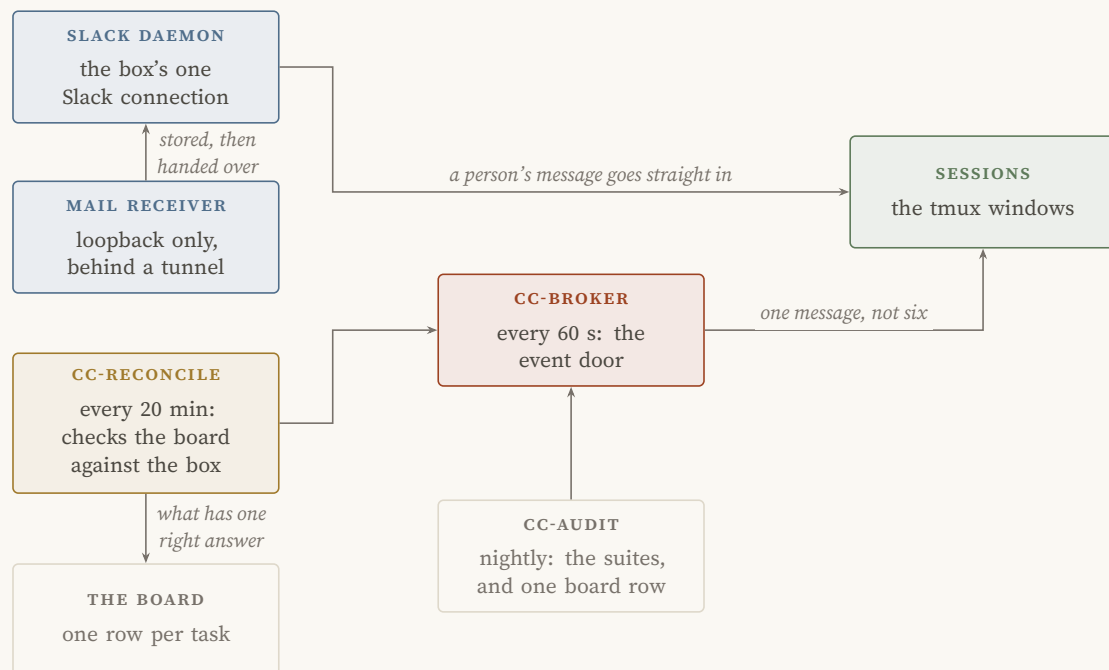
A few parts of the box run on their own, and none of them calls a model. They notice things, hand what they noticed to a session, and leave the judgement to it.

cc-reconcile is the only mechanical watcher left. Every twenty minutes it checks the board against the box, to see which tasks have a live session, which pull requests merged and which worktrees are dead. Where there is one right answer it writes that to the board, and where a person has to decide it hands the question to that project's session. It also carries the daily sweep and the owner's daily line.

cc-broker is the door those events come through. A session pays for a whole turn every time an event reaches it, however small, so the broker groups repeats and delivers them once. A message from a person never goes through it and cannot be held.

cc-audit runs at 03:30 UTC. It runs the suites, updates one board row when the default branch is failing, and writes the day's report. It fixes nothing. Six smaller timers do the rest. Every 60 s they drain messages and watch for a usage limit on the main model, every 2 min they sweep stuck dialogs, every 5 min they checkpoint member work and tally the spend, and every 15 min they publish.

Counts: distinct pull requests in `~/cc/state/land/queue.log` from 2026-09-01, excluding deploy-only jobs. Timers: `systemctl --user list-timers`, and `core/config/systemd-user/*.timer`.

FIGURE 5 *what runs with nobody there*

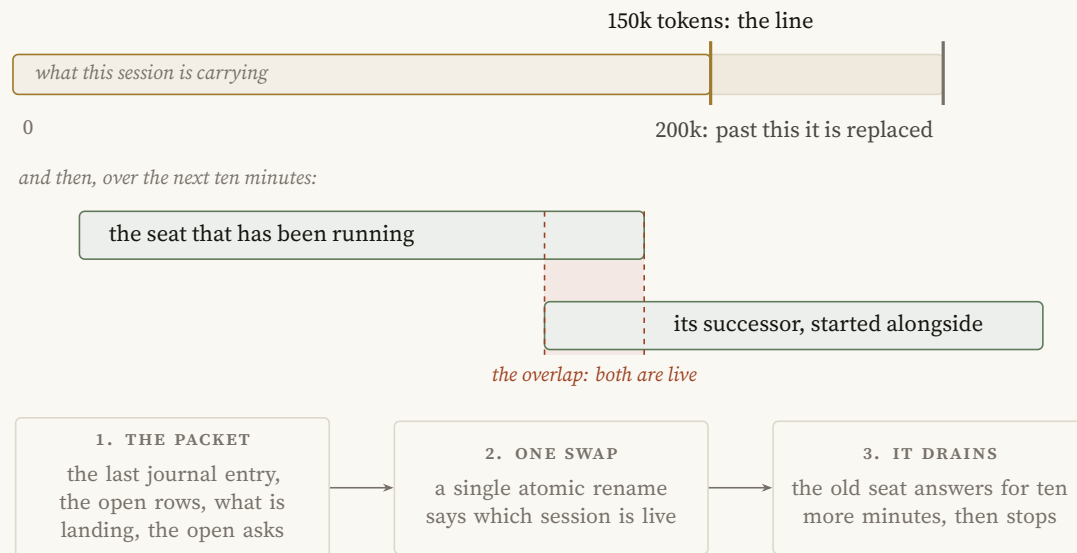
Slate boxes are doors in, ochre the watcher, rust the door every event passes, sage where a model runs, grey a timer and a record. Italic says what passes.

There used to be ten watcher processes. In the four days to 16 September they were folded into these two timers, which took 15,525 lines down to about 4,000, and neither one calls a model any more.

How a long-lived session hands over

A session that has been running all day is expensive, because every turn re-reads everything that came before it. So a session is replaced before it gets that heavy, and its replacement starts up while the old one is still answering.

Headers of core/bin/cc-reconcile, cc-broker and cc-audit for the no-model claim. The fold and its line counts: docs/scaffolding.md, measured at two named commits.

FIGURE 6 *the line, the overlap, and the three steps*

The bar is what one session carries. The sage bars are the old seat and its successor, the rust band the stretch when both are live, and grey boxes the three steps.

The line is a fixed number of tokens rather than a share of the window, so it is the same 150k whether the window holds 200k or a million. Past it, the session is nudged once an hour to write a dated journal entry, and it decides for itself when to hand over. Nothing on the box hands over for a session. Past 200k it is replaced whether it decided to or not.

Handing over costs about two million tokens, counting everything as input. That goes on seventy-odd tool calls by the new session as it finds its feet, and eighteen turns of the old one answering its questions. It is the price of keeping a session from carrying a day of conversation into every later turn.

Workers never hand over. Each round starts on a fresh context that reads the journal, so for a worker the journal is the handover, and it happens every time.

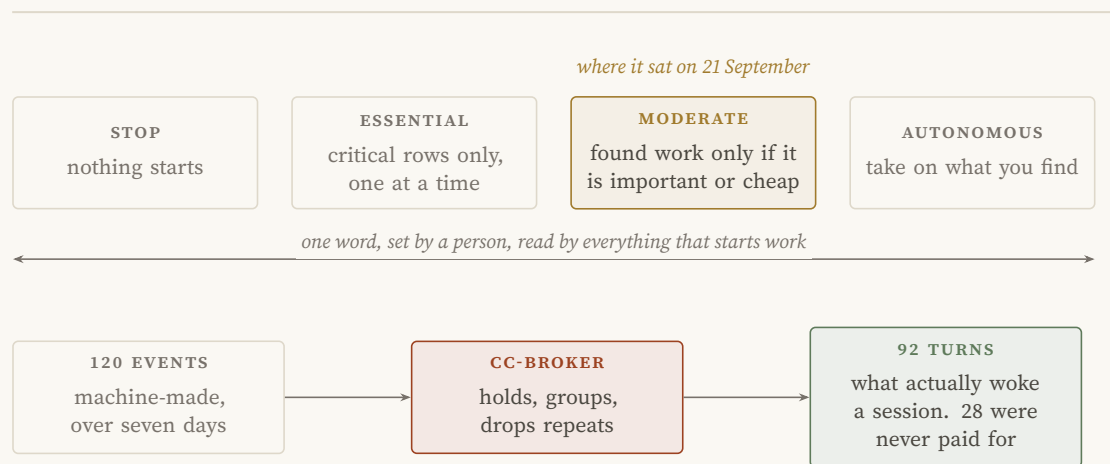
CC_CONTEXT_HANDOFF_TOKENS (default 150k) and CC_CONTEXT_ROTATE_TOKENS (200k) in `~/cc/config`; the overlap, the atomic rename and the ten-minute drain in `core/bin/cc-handoff` and `core/docs/DESIGN.md` §3–§4. The measured cost is recorded there too.

What it costs, and what holds it down

The money goes on context rather than on the code that gets written. Waking a session costs one turn, and a turn re-reads everything that session is carrying, so the cheapest saving is nearly always one fewer wake-up.

There are four rules, in order of priority: wake a session as seldom as possible, keep its context small, use the cheapest model that still holds up, and buy one review read of each change. A monitor fires only when something has finished or failed, and only once, because a line for every stage it passed through is the same information at several times the price.

FIGURE 7 *the one word, and what a week of events cost*



Top, the four spend tiers, ochre where it sat. Bottom, a week of machine-made events, through the rust broker, to the sage turns that woke a session.

How much work the box takes on is set by one word a person chooses, and everything that starts work reads it. It never stops the box answering the owner, doing a task he asked for, finishing a landing already past its gates, or writing to the board.

Models are picked by the job. Planning seats and orchestrators run on the strongest one, because their judgement is what the whole day rests on. Workers run one step down. A subagent doing one file and a dozen lines runs two steps down, and a member session runs there by default.

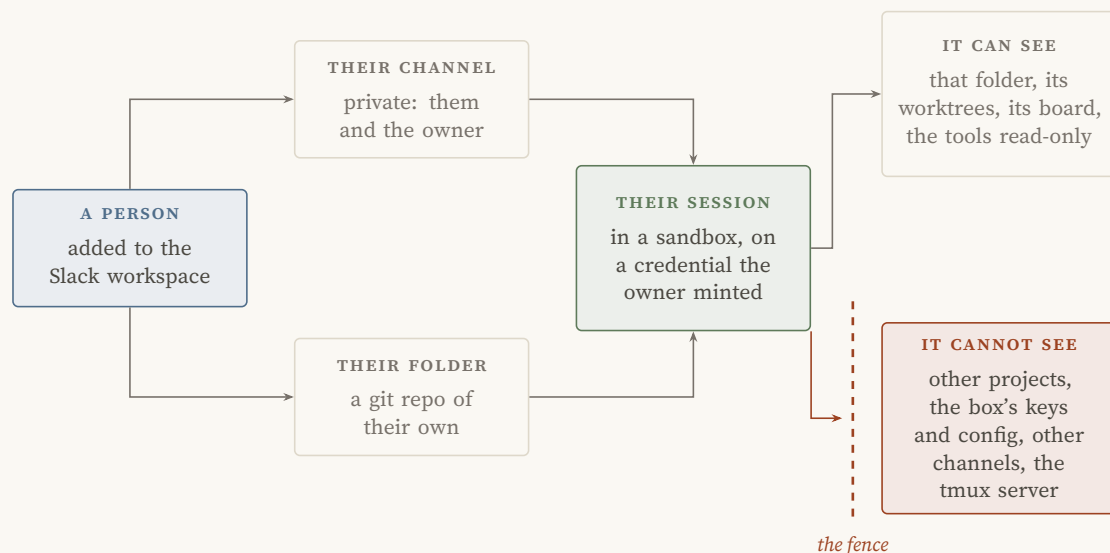
Tier: cc-tier (four words; CC_SPEND_TIER). Event and turn counts: cc-broker report --days 7, run 21 September. The four rules: config/spend-prompt.md. Model bands: ~/WORKING.md.

In the week to 21 September the box logged 2,150 wake-ups. Just under half of what those turns re-read was the work itself. The rest was the session's own standing context, things it had already read once, and handovers. That rest is what the four rules are aimed at.

Other people, and the fence

Someone other than the owner gets a channel, a folder and a session of their own, and nothing else on the machine.

FIGURE 8 *a member's workspace, and the fence around it*



Slate is the person, grey what they are given and what their session can see, sage their session and rust what it cannot see. The rust dashed line is the fence.

Their session starts inside a bubblewrap sandbox, on a filesystem put together for it. It can write to its own folder, it sees the box's tools but cannot change them, it holds one Claude credential the owner minted, and its view of Slack has only its own channels. Without that credential the workspace does not start at all.

WHAT THE COMMAND GATES DO NOT STOP

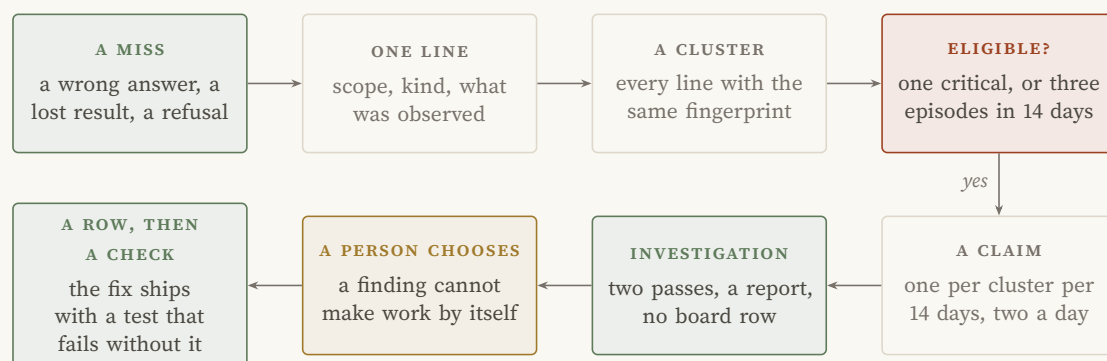
A second and softer layer sits on top. The command gates refuse a member session the things a filesystem boundary cannot express, such as starting a worker or re-wiring Slack. Those gates work from a list of what is forbidden, so they stop accidents and casual prompt injection but not someone determined to get through. The filesystem boundary is the part that holds.

A member workspace can merge its own pull requests once the owner gives it a GitHub token. That merge runs from a clone only the host can reach, runs the gates inside the member's own sandbox, skips the deploy step, charges the review to that workspace's daily budget, and stops outright on a change that touches the fence or a token. None of it touches the box itself.

What happens when it is wrong

Agents get things wrong. Every failure an agent admits to becomes one line in a ledger, and the ledger decides what gets looked into.

FIGURE 9 *from one miss to one new check*



Sage is an agent at work, grey the ledger, rust the eligibility rule and ochre the person who chooses.

The gate list per role: `core/bin/cc-guard`. The blocklist caveat is stated in `core/docs/DESIGN.md` §4. Member landing: MEMBERS in `core/bin/cc-land`.

What matters is whether the same thing goes wrong twice.

THE ONE CLAIM THIS DOCUMENT MAKES

738

records in the ledger, 730 of them confirmed

106

clusters, meaning lines that say the same thing

22

of those clusters eligible for an investigation today

The grouping is worked out each time the ledger is read and never stored, so a cluster is only ever what the lines actually say.

WHAT THE LEDGER DOES NOT DO

Two limits keep this from running away with itself. A cluster can be claimed once in fourteen days, and at most two investigations start in a rolling day. An investigation writes a report and posts it, and that is all it does. Someone has to choose, on the record, before a finding turns into work, because six reviews in six days once turned into work on their own.

A second habit sits next to the ledger. Before a session says anything about a permission, a merge, a running process or what a task did, it has to read the record instead of trusting its memory. Fifteen recorded misses led to that rule, and in twelve of them the answer was sitting on disk.

Counts: `cc-failures metrics`, run 21 September. The eligibility rule, the claim limits and the fingerprint: `core/bin/cc-failures` and `docs/design-failure-noticing-2026-09-11.md`. The reading habit and its fifteen misses: `core/bin/cc-lib` and `docs/design-knowledge-library-2026-09-11.md`.